1. Descripción general

Los sistemas operativos deben administrar la memoria principal para permitir que diferentes procesos puedan ejecutarse, procurando aprovechar los recursos disponibles de manera eficiente. Una de las técnicas clásicas de administración de memoria consiste en dividir la memoria principal en **particiones de tamaño fijo**. Cada partición puede contener un único proceso y, una vez establecidas las particiones, sus tamaños no pueden modificarse durante la ejecución del sistema. En esta tarea se desarrollará un **simulador de administración de memoria mediante particiones fijas**, incorporando dos estrategias de selección de particiones inspiradas en los algoritmos *First Fit* (primer ajuste) y *Best Fit* (mejor ajuste).

Aunque estos algoritmos suelen estudiarse en el contexto de asignación de memoria variable, en esta tarea se adaptarán para seleccionar, entre las particiones fijas disponibles, aquella donde se colocará un proceso.

El simulador deberá permitir la llegada de procesos, su asignación a memoria, su finalización y la liberación de las particiones ocupadas.

2. Objetivo general

Implementar un programa que simule la administración de memoria principal mediante particiones fijas, utilizando estrategias de asignación que permitan analizar el aprovechamiento de la memoria y la fragmentación interna.

3. Requerimientos funcionales

3.1. Configuración inicial de memoria

El programa permitirá configurar una memoria principal dividida en una cantidad determinada de particiones.

Cada partición tendrá:

- Un identificador único.
- Un tamaño expresado en KB.
- Un estado: libre u ocupada.
- El identificador del proceso que la ocupa, cuando corresponda.

Las particiones podrán tener **diferentes tamaños entre sí**, pero su tamaño individual permanecerá fijo durante toda la simulación.

Por ejemplo, una configuración válida sería:

Partición	Tamaño
1	120 KB
2	300 KB
3	200 KB
4	500 KB

La memoria simulada tendrá una capacidad total de 1120 KB, distribuida en cuatro particiones.

No se permitirá crear nuevas particiones, dividir una partición existente ni modificar su tamaño durante la simulación.

3.2. Administración de procesos

El programa deberá permitir registrar procesos, cada uno con la siguiente información:

- Identificador único del proceso.
- Tamaño de memoria requerido en KB.
- Estado del proceso: esperando, en memoria o finalizado.
- Partición asignada, cuando corresponda.

Cada proceso debe ocupar una única partición. No podrá distribuirse entre varias particiones, aunque la suma de sus espacios libres sea suficiente para almacenarlo.
Si un proceso requiere más memoria que la partición de mayor tamaño del sistema, deberá ser rechazado por imposibilidad de asignación.

3.3. Estrategias de asignación

El simulador debe implementar las siguientes estrategias. El usuario podrá seleccionar cuál utilizar.

A. First Fit (primer ajuste)

El sistema recorrerá las particiones en el orden en que fueron definidas.
Se asignará el proceso a la **primera partición libre cuyo tamaño sea suficiente** para contenerlo.
Por ejemplo, si un proceso requiere 180 KB y existen particiones libres de 120, 300, 200 y 500 KB, se seleccionará la partición de 300 KB.

B. Best Fit (mejor ajuste)

El sistema buscará entre todas las particiones libres aquella que tenga el **menor tamaño suficiente** para almacenar el proceso.
Por ejemplo, utilizando la misma configuración, un proceso de 180 KB se asignará a la partición de 200 KB.
Si existen dos o más particiones igualmente adecuadas, se seleccionará aquella con el menor identificador.
Importante: En ambos algoritmos, el proceso ocupa completamente la partición en términos de disponibilidad, aunque no utilice toda su capacidad. El espacio sobrante se considera fragmentación interna y no puede asignarse a otro proceso.

3.4. Cola de espera

Si un proceso no puede colocarse en ninguna partición disponible, deberá ingresar a una cola de espera.
El programa deberá:

1. Mantener los procesos pendientes en orden de llegada.
2. Mostrar cuáles procesos están esperando memoria.
3. Revisar la cola cuando se libere una partición.
4. Intentar asignar los procesos pendientes utilizando el algoritmo seleccionado.

La revisión de la cola se realizará en orden de llegada. Cuando un proceso no pueda asignarse, permanecerá esperando, pero el sistema podrá continuar revisando los siguientes procesos para aprovechar otras particiones disponibles.

3.5. Finalización de procesos

El usuario podrá indicar que un proceso que se encuentra en memoria ha terminado.
Cuando esto ocurra:

- Se liberará la partición ocupada.
- Se actualizará el estado del proceso a finalizado.
- Se revisará la cola de espera para determinar si alguno de los procesos pendientes puede ingresar a memoria.
- Se mostrará el nuevo estado de la memoria.

No se permitirá finalizar procesos inexistentes o que no se encuentren actualmente en memoria.

3.6. Visualización del estado de memoria

El programa deberá mostrar la situación actual de cada partición mediante una tabla que incluya:

Partición	Tamaño	Estado	Proceso	Memoria no utilizada
1	120 KB	Ocupada	P2	10 KB
2	300 KB	Ocupada	P3	50 KB
3	200 KB	Ocupada	P1	20 KB
4	500 KB	Libre	—	500 KB

La última columna representa fragmentación interna en las particiones ocupadas. En las particiones libres representa capacidad disponible, no fragmentación interna.
Adicionalmente, se mostrará la lista de procesos pendientes en la cola de espera.

4. Cálculo de estadísticas

El programa deberá calcular y mostrar, como mínimo:

1. **Memoria total:** suma de los tamaños de todas las particiones.
2. **Memoria utilizada por procesos:** suma de los tamaños de los procesos que se encuentran en memoria.
3. **Memoria libre:** suma de las capacidades de las particiones desocupadas.
4. **Fragmentación interna:** suma de las diferencias entre el tamaño de cada partición ocupada y el tamaño del proceso asignado.
5. **Porcentaje de utilización efectiva:** memoria utilizada por procesos dividida entre memoria total, multiplicada por 100.
6. **Cantidad de procesos en espera:** procesos que aún no han logrado ingresar a memoria.

Debe cumplirse la siguiente relación:
Memoria total = Memoria utilizada por procesos + Fragmentación interna + Memoria libre.

5. Ejemplo de ejecución

Considere la siguiente configuración:

Partición	Tamaño
1	120 KB
2	300 KB
3	200 KB
4	500 KB

Los siguientes procesos llegan en el orden indicado: (son introducidos por el usuario)

Proceso	Memoria requerida
P1	180 KB
P2	110 KB
P3	250 KB

Resultado utilizando First Fit:

Proceso	Partición asignada	Fragmentación interna
P1	2 (300 KB)	120 KB
P2	1 (120 KB)	10 KB
P3	4 (500 KB)	250 KB

Fragmentación interna total: **380 KB**.

Resultado utilizando Best Fit:

Proceso	Partición asignada	Fragmentación interna
P1	3 (200 KB)	20 KB
P2	1 (120 KB)	10 KB
P3	2 (300 KB)	50 KB

Fragmentación interna total: **80 KB**.

En este escenario, Best Fit produce una menor fragmentación interna que First Fit. Sin embargo, esto no significa que necesariamente produzca mejores resultados en todos los escenarios.

6. Interfaz del programa

El simulador deberá ofrecer un menú que permita ejecutar, como mínimo, las siguientes operaciones:

1. Configurar las particiones de memoria.
2. Seleccionar estrategia de asignación: First Fit o Best Fit.
3. Crear un nuevo proceso e intentar asignarlo.
4. Finalizar un proceso que se encuentre en memoria.
5. Consultar el estado de las particiones.
6. Consultar la cola de espera.
7. Mostrar estadísticas de utilización de memoria.
8. Reiniciar la simulación.
9. Salir del programa.

Se permite utilizar una interfaz de consola. No se requiere interfaz gráfica.

La configuración de particiones y la selección de estrategia deberán realizarse antes de comenzar la asignación de procesos. No será necesario admitir cambios de estrategia durante una simulación activa.

7. Restricciones técnicas

- La solución debe representar las particiones y los procesos utilizando estructuras de datos apropiadas.
- Los algoritmos First Fit y Best Fit deberán implementarse por separado.
- No se permitirá utilizar bibliotecas que implementen directamente los algoritmos solicitados.
- Se deberán utilizar funciones o métodos para separar las responsabilidades principales.
- El programa deberá validar los tamaños de las particiones y de los procesos para evitar valores negativos o iguales a cero.
- Se deberá impedir la existencia de dos procesos activos con el mismo identificador.
- No se permitirá mover procesos entre particiones una vez asignados.
- No se implementará compactación de memoria ni intercambio de procesos con disco.

8. Pruebas obligatorias

Los estudiantes deberán ejecutar y documentar al menos los siguientes escenarios:

Escenario	Situación que se debe probar
1	Un proceso cabe exactamente en una partición libre
2	Un proceso cabe en varias particiones de diferentes tamaños
3	Un proceso no cabe en ninguna partición libre
4	Un proceso requiere más memoria que cualquier partición existente
5	Todas las particiones están ocupadas
6	La finalización de un proceso permite asignar uno que estaba esperando
7	La suma de la memoria libre es suficiente, pero ninguna partición individual puede alojar el proceso

8	First Fit y Best Fit producen asignaciones diferentes para la misma secuencia de procesos

9. Entregables

Se deberá entregar:

A. Código fuente

Programa completo y funcional, organizado y comentado apropiadamente.

B. Documento de análisis (PDF)

Un documento breve que contenga:

1. Descripción de las estructuras de datos utilizadas.
2. Explicación de la implementación de First Fit y Best Fit.
3. Evidencias de las pruebas realizadas.
4. Comparación de los resultados obtenidos mediante los dos algoritmos.
5. Respuestas a las siguientes preguntas de análisis:
    - ¿Por qué First Fit y Best Fit pueden aplicarse a particiones fijas aunque normalmente se estudien en el contexto de particiones variables?
    - ¿Por qué el espacio sobrante dentro de una partición ocupada no puede utilizarse para otro proceso?
    - ¿En qué circunstancias Best Fit puede producir una mayor fragmentación interna total que First Fit?
    - ¿Es posible tener suficiente memoria libre total y aun así no poder asignar un proceso? Explique.
    - ¿Cuál sería el impacto de permitir que los procesos se distribuyan entre varias particiones? ¿Qué cambios implicaría para el modelo de administración de memoria?

10. Criterios de evaluación

Criterio	**Porcentaje**
Modelado y configuración de particiones fijas	15%
Implementación correcta de First Fit y Best Fit	25%
Gestión de procesos, liberación y cola de espera	25%
Estadísticas y fragmentación interna	15%
Validaciones y pruebas funcionales	10%
Documento de análisis y calidad del código	10%
Total	**100%**

11. Consideraciones finales

El propósito de esta tarea no es únicamente lograr que los procesos sean asignados a memoria, sino comprender las limitaciones que introduce el uso de particiones de tamaño fijo.

Se espera que el estudiante observe cómo **la estrategia de asignación, los tamaños de las particiones y el orden de llegada de los procesos** afectan el aprovechamiento de la memoria.

El programa deberá permitir experimentar con diferentes configuraciones para identificar escenarios en los que una estrategia ofrece ventajas sobre la otra.